```
{
    class Program
    {
        [STAThread] // <=== Required for WPF.
        static void Main(string[] args)
        {
            for // <=== Only one statement.
            (
                // Use anonymous class as "iterator" variable. Need to use it as a
                // wrapper because C#'s for will not allow me to declare variables
                // of multiple types in the for initializer.
                //
                var i = new // <=== Call it i for convention's sake. :-)
                {
                    //
                    // Gratuitous use of XElement to construct an XML element with multiple attributes.
                    //
                    xmlAnim = new XElement
                    (
                        "DoubleAnimation",
                        new XAttribute("Storyboard.TargetName", "myVB"),
                        new XAttribute("Storyboard.TargetProperty", "(Viewbox.Opacity)"),
                        new XAttribute("From", "1"),
                        new XAttribute("To", "0"),
                        new XAttribute("Duration", "0:0:2"),
                        new XAttribute("AutoReverse", "True"),
                        new XAttribute("RepeatBehavior", "Forever")
                    ),
                    totalHours = new List<float>(),
                    user = System.Environment.GetEnvironmentVariable("USERNAME"),
                    ctx = new NorthwindDataContext(),
                    rnd = new Random(),
                    //
                    // Use generic types.
                    //
                    categories = new List<string> { "Meetings", "Coding", "Debugging", "Web surfing" }.GetEnumerator() as IEnumerator<string>, // <=== Thanks, 
Mark!
                    app = new Application() { ShutdownMode = ShutdownMode.OnLastWindowClose },
                    mutex = new Mutex(false, "myMutex")
                }
                //   *
                //  *
            ;   // **************** FIRST SEMICOLON ****************
                //  *
                //   *

                //
                // Use Linq to Objects
                //
                ((from hours in i.totalHours select hours).Sum()) <= 40d &&
                  i.categories.MoveNext() ? true :
                  //
                  // Anonymous method.
                  //
                  new Func<bool>
                  (
                    //
```

```
// Lamda expression taking no parameters is the body of the anonymous method
//
() =>
Double.IsNaN // <=== KLUDGE!, but gets us an arbitrary false back (because Application.Run returns a valid number),
             // which we want to return after all the side effects below.
(
    //
    // Use WPF at run-time.
    //
    i.app.Run
    (
        new Window()
        {
            Title = "Only One STATEMENT (but two semicolons) Demo",
            Width = 400,
            Height = 300,
            Name = "myWindow",
            //
            // Build and load XAML at run-time (cast not strictly necessary).
            //
            Content = (Viewbox)XamlReader.Load // <=== Second attempt at async here using AsyncLoad. Can't attach LoadCompleted event.
            (
                new MemoryStream
                (
                    Encoding.UTF8.GetBytes // <=== Thanks, Mark!
                    (
                        @"<Viewbox xmlns='http://schemas.microsoft.com/winfx/2006/xaml/presentation' " +
                            @"xmlns:x='http://schemas.microsoft.com/winfx/2006/xaml' " + // <=== Thanks, Mike!
                            @"x:SynchronousMode='Async' " +
                            @"Name='myVB' Width='400' Height='300' >" +
                            @"<Label Name='myLabel' " +
                                @"VerticalAlignment='Center' HorizontalAlignment='Center' " +
                                @"VerticalContentAlignment='Center' HorizontalContentAlignment='Center' " +
                                //
                                // Binding to other element
                                //
                                @"Width='{Binding ElementName=myVB, Path=Width}' " +
                                @"Height='{Binding ElementName=myVB, Path=Height}' " +
                                @"FontWeight='Bold' FontSize='16pt' " +
                            @">" +
                                @"Total hours: " +
                                //
                                // Linq to Objects.
                                //
                                (from hours in i.totalHours select hours).Sum().ToString() +
                                @"<Label.Background>" +
                                    @"<ImageBrush ImageSource='Bliss.jpg' />" +
                                @"</Label.Background>" +
                            @"</Label>" +
                            //
                            // Stupid animation.
                            //
                            @"<Viewbox.Triggers>" +
                                @"<EventTrigger RoutedEvent='Viewbox.Loaded' SourceName='myVB'>" +
                                    @"<BeginStoryboard>" +
                                        @"<Storyboard>" +
                                            //
```

```
                                                    // REALLY gratuitous use of that gratuitous XElement!
                                                    //
                                                    i.xmlAnim.ToString() +
                                                @"</Storyboard>" +
                                            @"</BeginStoryboard>" +
                                        @"</EventTrigger>" +
                                    @"</Viewbox.Triggers>" +
                                @"</Viewbox>"
                            )
                        ),
                        //
                        // Black magic needed to make relative path in ImageBrush.ImageSource to work.
                        //
                        new ParserContext() { BaseUri = new Uri("pack://siteoforigin:,,,") } // <=== Thanks, Google!
                    )
                }
            )
        )
    ).Invoke() // <=== Call our anonymous method.
   //   *
   //  *
;  // **************** LAST SEMICOLON ****************
   //  *
   //   *

    i.totalHours.Add(i.rnd.Next(10, 20)),
    i.mutex.WaitOne(),
    //
    // Use Linq to SQL and object initializers.
    //
    i.ctx.TimeCards.InsertOnSubmit
    (
        new TimeCard()
        {
            Category = i.categories.Current,
            Hours = i.totalHours.Last(),
            UserId = i.user,
            Year = System.DateTime.Now.Year,
            Week = ((System.DateTime.Now.DayOfYear / 7) + 1)
        }
    ),
    i.mutex.ReleaseMutex(),
    //
    // How 'bout some gratuitous threading using a lamda expression as an anonymous function
    // and using reflection to call the methods in the lamda expression (because two of the methods
    // return void, it is hard to use them in an expression otherwise).
    //
    new Thread
    (
        new ThreadStart
        (
            () =>
            new StringBuilder
            (
                i.mutex.GetType().GetMethod("WaitOne", new Type[] {}).Invoke(i.mutex, new object[] {}).ToString().Length +
                (int)(i.ctx.GetType().GetMethod("SubmitChanges", new Type[] {}).Invoke(i.ctx, null) == null ? 0 : 1) +
                (int)(i.mutex.GetType().GetMethod("ReleaseMutex", new Type[] {}).Invoke(i.mutex, new object[] {}) == null ? 0 : 1)
```

```
                        ).ToString()
                    )
                ) { IsBackground = true }.Start()
            )
            { /* Intentionally empty - coulda just closed the for statement with a semicolon, but that woulda added another semicolon! :-) */ }
        }
    }
}
```